

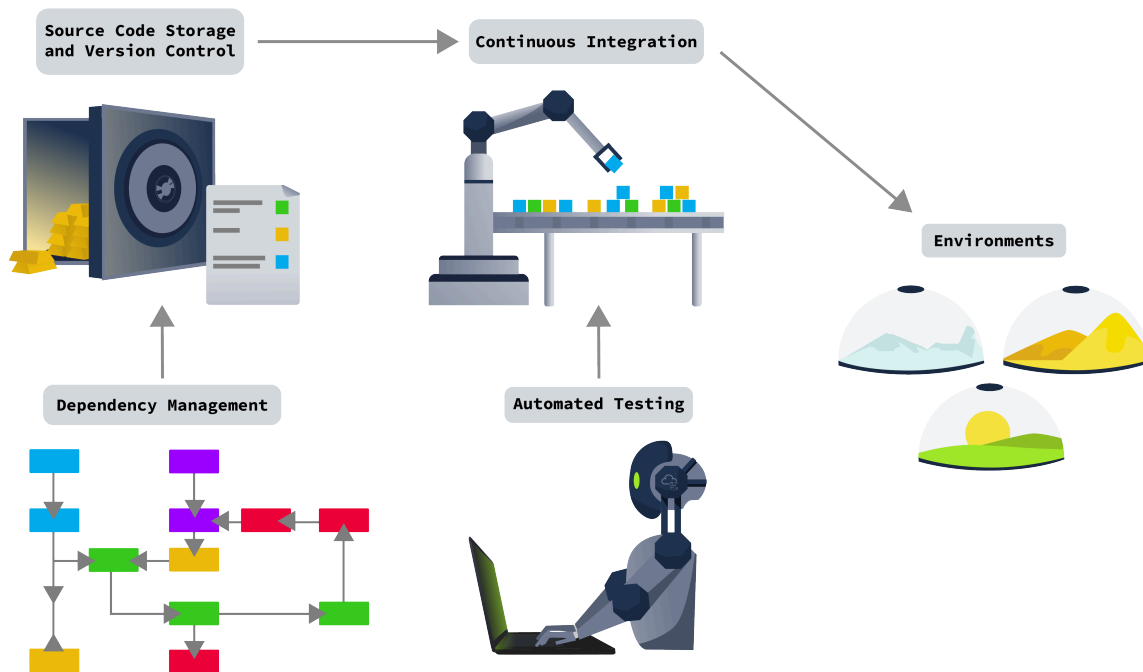
DevSecOps 2-1-Intro-to-Pipeline-Automation

Task 1 - Introduction

Automation in SDLC and DevOps accelerates production but shifts security risks toward pipelines rather than manual workstations.

Task 2 - DevOps Pipelines Explained

The diagram below shows what a typical pipeline can look like, as well as the software that could be used for this purpose:



For each of these items, we will look at what they are, the common tools used for them, an introduction to their security, and a case study of what can happen when security fails.

Answer the questions below

| Where in the pipeline is our end product deployed?

Environments

Task 3 - Source Code and Version Control

Source Code Storage: We need to consider several things when deciding where to store our code:

- How can we perform access control for our source code?
- How can we make sure that changes made are tracked?
- Can we integrate our source code storage system with our development tools?
- Can we store and actively use multiple different versions of our source code?
- Should we host our source code internally, or can we use an external third party to host our code?

The answers to these questions will help us choose the correct source code storage solution for our project.

Version Control: We need version control for two main reasons:

- We are often integrating new features in our software. Modern development approaches, such as Agile, means we are constantly updating our code. To keep all of these updates in check, we need version control.
- An entire development team is working on the code, not just one developer. To ensure that we can integrate the changes from multiple developers, version control is required.

Version control allows us to keep multiple versions of the code. This can be the specific version each developer is working on, but it can also be completely different versions of our application, including minor and major versions.

Common Tools: The two most common source code storage and version control systems are Git and SubVersion (SVN). Git is a distributed source control tool, meaning that each contributor will have their own copy of the source code. On the other hand, SVN is a centralised source control tool, meaning the control of the repo is managed centrally. GitHub is by far the largest provider of Internet hosting for software development and version control using Git. You can create a GitHub

account and use that to manage your source code repositories (repo). However, you could also host your own git server using software such as Gitlab. For SVN, the two most popular tools are TortoiseSVN and Apache SVN. However, it should be noted that source code storage solutions such as Gitlab provide much more features than simple storage and version control.

Security Considerations:

- **Access & Tracking:** Secure source code through strict authentication and access control while maintaining a version history to enable rollbacks.
- **Separation of Secrets:** Never store sensitive credentials or connection strings in source code repositories, as version histories permanently retain past exposures even after deletion.

Case Study: Git Never Forgets:

- **Permanent Version History:** Git retains every historical commit, meaning deleted files or removed secrets remain accessible in past versions of the repository.
- **Vulnerability to Scanners:** Attackers can use automated tools like GittyLeaks to scan historical commits and extract sensitive credentials, even if they were removed in later updates.

Answer the questions below

| Who is the largest online provider of Git?

Github

| What popular Git product is used to host your own Git server?

Gitlab

| What tool can be used to scan the commits of a repo for sensitive information?

GittyLeaks

Task 4 - Dependency Management

Managing pre-written libraries and SDK dependencies that make up the vast majority of application code.

External Dependencies: Publicly available libraries and SDKs hosted on external dependency managers like PyPi, NuGet, and Gems.

Internal Dependencies: Proprietary libraries and SDKs developed and maintained internally to be shared across an organization's applications.

There are different security concerns for internal and external dependencies:

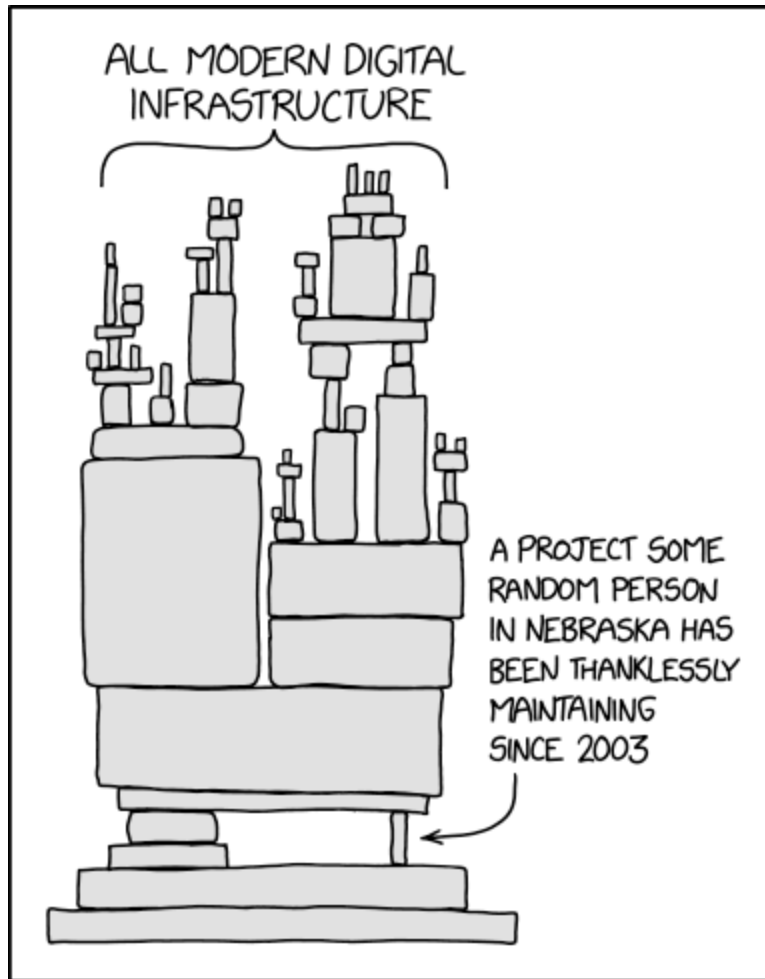
Internal	External
Libraries can often become legacy software since they no longer receive updates or the original developer has left the company.	Since we do not have full control over the dependency, we must perform due diligence to ensure that the library is secure.
The security of the package manager is our responsibility for internal libraries.	If a package manager or content distribution network (CDN) is compromised, it could lead to a supply chain attack.
A vulnerability in an internal library could affect several of our applications since it is used in all of them.	External libraries can be researched by attackers to discover Oday vulnerabilities. If such a vulnerability is found, it could lead to the compromise of several organisations at the same time.

Common Tools: Package managers like PyPi, NuGet, and Gems handle external libraries, while tools like JFrog Artifactory and Azure Artifacts manage internal, organization-wide dependencies.

Security Considerations: External dependencies introduce third-party code outside your direct control, making it difficult to track and leaving applications vulnerable if flaws exist in those libraries.

Case Study: Log4Shell

A Oday vulnerability was discovered in Log4j dependency in 2021 called Log4Shell. Log4j is a Java-based logging utility. It is part of the Apache Logging Services, a project of the Apache Software Foundation. The vulnerability could allow an unauthenticated attacker to gain remote code execution on a system that makes use of the logger. The true issue? This small little dependency was used almost literally everywhere, as shown by this [XKCD\(opens in new tab\)](#) cartoon:



Because this dependency was embedded across countless global products, the vulnerability exposed the catastrophic scale of risk inherent in third-party software supply chains.

Answer the questions below

| What do we call the type of dependency that was created by our organisation?

Internal

| What type of dependency is JQuery? (Internal/External)

External

| What is the name of Python's public dependency repo?

PyPi

| What dependency Oday vulnerability set the world ablaze in 2021?

Log4j

Task 5 - Automated Testing

Replacing tedious manual testing with automated processes integrated directly into modern deployment pipelines.

Unit Testing: Tests isolated, small parts of an application to verify functionality and acts as a CI/CD quality gate, though it generally excludes security checks.

Integration Testing: Evaluates how multiple application components work together within the CI/CD pipeline, including regression testing to protect existing features from new updates, but typically focuses on functionality rather than security.

Security Testing: There are two primary types of automated security testing.

SAST: Static Application Security Testing (SAST) works by reviewing the source code of the application or service to identify sources of vulnerabilities. SAST tools can be used to scan the source code for vulnerabilities. This can be integrated into the development process to already highlight potential issues to developers as they are writing code. We can also integrate this into the CI/CD process. Not as quality gates, but as security gates, preventing the pipeline from continuing if the SAST tool still detects vulnerabilities that have not been flagged as false positives.

DAST: Dynamic Application Security Testing (DAST) is similar to SAST but performs dynamic testing by executing the code. This allows DAST tools to detect additional vulnerabilities that would not be possible with just a source code review. One method that DAST tools use to find additional vulnerabilities, such as Cross Site Scripting (XSS), is by creating sources and sinks. When a DAST tool provides input to a field in the application, it marks it as a source. When data is returned by the application, it looks for this specific parameter again and, if it finds it, will mark it as a sink. It can then send potentially malicious data to the source and, depending on what is displayed at the sink, determine if there is a vulnerability such as XSS. Similar to SAST, DAST tools can be integrated into the CI/CD pipeline as security gates.

Penetration Testing: Sadly, SAST and DAST tools cannot fully replace manual testing, such as penetration tests. There have been significant advancements in

automated testing and even in some cases, these techniques were combined with more modern approaches to create new testing techniques such as Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP). However, the main issue remains that these tools, including these modern testing techniques, do not perform well against contextual vulnerabilities. Take the process flow of a payment, for example. A common vulnerability is when part of the process can be bypassed, for example, the credit card validation step. This is an easy test case to perform manually, but since it requires context, even DAST tooling will find it hard to discover the bypass. Similarly, business logic and access control flaws are hard to discover using automated tools, whereas manual testing can discover them fairly quickly. It is not that automated tooling will never be able to find these flaws, it is simply more cost-effective to use manual testing.

Common Tools: There are several common tools that can be used for automated testing. Both [GitHub\(opens in new tab\)](#) and [Gitlab\(opens in new tab\)](#) have built-in SAST tooling. Tools such as [Snyk\(opens in new tab\)](#) and [Sonarqube\(opens in new tab\)](#) are also popular for SAST and DAST.

Case Study: She cannae take any more captain, She's gonna blow! A common issue with SAST and DAST tooling is that the tool is simply deployed into the pipeline, even simply for a Proof-of-Concept (PoC). However, you need to take several things into consideration:

- Performance cost
- Integration points
- Calibration of results
- Quality and security gate implementation

Performance Impact: Initial proofs-of-concept and continuous security scans can severely degrade source code control performance, blocking developers from staging or pushing critical pre-release commits.

Infrastructure Strain: High-velocity repositories receiving hundreds of daily commits can experience severe latency bottlenecks if automated security gates evaluate every merge request.

Strategic PoC Deployment: Schedule initial tool evaluations during off-peak hours to prevent operational disruptions while maintaining a representative test

environment.

Answer the questions below

| What type of tool scans code to look for potential vulnerabilities?

SAST

| What type of tool runs code and injects test cases to look for potential vulnerabilities?

DAST

| Can SAST and DAST be used as a replacement for penetration tests? (Yea,Nay)

Nay

Task 6 - Continuous Integration and Delivery

In modern pipelines, software isn't manually moved between different environments. Instead, an automated process can be followed to compile, build, integrate, and deploy new software features. This process is called CI/CD.

Note: The evolution of CI/CD terminology from development-focused agile processes to comprehensive, end-to-end delivery and monitoring pipelines.

CI/CD: Continuous Integration: Regularly building and testing new features alongside the current application as they are developed, rather than waiting until the end of the cycle. **CI/CD Pipeline:** An automated workflow designed to streamline the integration, testing, and deployment of software changes.

- Starting Trigger - The action that kicks off the pipeline process. For example, a push request is made to a specific branch.
- Building Actions - Actions taken to build both the project and the new feature.
- Testing Actions - Actions that will test the project to ensure that the new feature does not interfere with any of the current features of the application.
- Deployment Actions - Should a pipeline succeed, the deployment actions detail what should happen with the build. For example, it should then be pushed to the Testing Environment.

- **Delivery Actions** - As CI/CD processes have evolved, the focus is now no longer just on the deployment itself, but all aspects of the delivery of the solution. This includes actions such as monitoring the deployed solution.

CI/CD pipelines require build-infrastructure to execute the actions of these elements. We usually refer to this infrastructure as build orchestrators and agents. A build orchestrator directs the various agents to perform the actions of the CI/CD pipelines as required. These CI/CD pipelines are usually where the largest portion of automation can be found. As such, this is usually the largest attack surface and the biggest chance for misconfigurations to creep in.

Common Tools: GitHub and Gitlab provide CI/CD pipeline capabilities and are quite popular to use. GitHub provides build agents, whereas Gitlab provides a Gitlab runner application that can be installed on a host to make it a build agent. For more complex builds, build orchestrator software such as Jenkins can be used. We will explore these tools and their common misconfigurations in later rooms.

Case Study: A tangle between Dev and Prod

One common misconfiguration with CI/CD pipelines is using the same build agents for both Development (DEV) and Production (PROD) builds. This creates an interesting problem since most developers will have access to the starting trigger for a DEV build but not a PROD build. If one of these developers were compromised, an attacker could leverage their access to cause a malicious DEV build that would compromise the build agent. This would not be a big issue if the build agent was just used for DEV builds. However, since this agent is also used for PROD builds, an attacker could just persist on this build agent until a PROD build is actioned to inject their malicious code into the build, which would allow them to compromise the production build of the application.

Answer the questions below

| What does CI in CI/CD stand for?

Continuous Integration

| What does CD in CI/CD stand for?

Continuous Delivery

What do we call the build infrastructure element that controls all builds?

Build Orchestrator

What do we call the build infrastructure element that performs the build?

Build Agent

Task 7 - Environments

Most pipelines have several environments. Each of these environments has a specific use case, and their security posture often differs. Let's take a look at some of the common ones:

Environment	Description	Stability	Security Posture	May it contain customer data?
DEV - Development	The DEV environment is the playground for developers. This environment is the most unstable as developers are continuously pushing new code and testing it. From a security standpoint, this environment has the weakest security. Access control is usually laxer, and developers often have direct access to the infrastructure itself. The likelihood of the development	Unstable	Weakest	No

	environment being compromised is high, but if there is adequate segregation, the impact of such a compromise should be low.			
UAT - User Acceptance Testing	The UAT environment is used to test the application or select features before they are pushed to production. These include unit tests that ensure the developed feature behaves as expected. This can (and should) include security tests as well. Although this environment is more stable than DEV, it can often still be fairly unstable. Similarly, certain security hardening controls would have been introduced for UAT, but it is still not as hardened as PreProd or PROD.	Semi-Stable	Second Weakest	No
PreProd - Pre-Production	The PreProd environment is	Stable	Second Strongest	No

	used to mimic production without actual customer/user data. This environment is kept stable and used to perform the final tests before the new feature is pushed to production. From a security standpoint, PreProd's security should technically mirror PROD. Although, this is not always the case.			
PROD - Production	The PROD environment is the most sensitive. This is the current active environment that serves users or customers. To ensure that our users have the best experience, this environment must be kept stable. No updates should be performed here without proper change management. To enforce this, the security of this	Stable	Strongest	Yes

	environment is the strongest. Only a select few employees or services will have the ability to make changes here. Furthermore, since we may have "malicious" users, the security has to be hardened to prevent outsider threats as well.			
DR/HA - Disaster Recovery or High Availability	Depending on the criticality of the system, there may be a DR or HA environment. If the switchover is instantaneous, it is usually called a HA environment. This is often used for critical applications such as Online Banking, where the bank has to pay large penalties if the website goes down. In the event where some (but still small) downtime is allowed, the environment is called a DR environment, meant to be used	Stable	Strongest	Yes

	to recover from a disaster in production. DR and HA environments should be exact mirrors of PROD in both stability and security.			
--	--	--	--	--

Other Notable Environments: There are some other environments that you may hear about when talking about DevOps.

Green and Blue Environments: Uses two identical production instances where Blue runs the current version and Green runs the update, allowing instant traffic switching and rapid rollbacks if issues arise.

Canary Environments: Gradually shifts a small percentage of users (e.g., 10%) to a new production environment, incrementally scaling up until 100% migration if stability is maintained.

Common Tools: Leverages virtualization, containerization, and Infrastructure as Code (IaC) via tools like Terraform, Vagrant, Docker, and Kubernetes to manage environments dynamically.

Security Considerations: Requires strict protection of production-adjacent infrastructure to prevent vulnerabilities from being exploited to gain control of hosts and applications. This hardening process usually requires things like the following:

- Removing unnecessary services
- Updating the host and applications
- Using a firewall to block unused ports

Case Study - Developer Bypasses in PROD: One of the common issues that can happen with different environments is that often things that should stay in DEV, don't. Develop bypasses are common in DEV environments for features like the following:

- Multi-factor authentication

- CAPTCHAs
- Password resets
- Login portals

Developer bypasses allow developers to quickly test different application features by bypassing time-consuming features such as MFA prompts. A common example is having a specific One-Time Pin (OTP) code that is always accepted, regardless of the OTP code that is sent by the application. However, if there is inadequate sanitisation of these bypasses before the application is moved to the next environment, it could lead to a developer bypass making its way all the way into PROD. That OTP bypass? It could now be leveraged by an attacker to bypass MFA and compromise user accounts. This is why environments must be segregated, and similar to quality gates, security gates must be implemented to ensure a clean application is moved to the next environment.

Answer the questions below

Which environment usually has the weakest security configuration?

DEV

Which environment is used to test the application?

UAT

Which environment is similar to PROD but is used to verify that everything is working before it is pushed to PROD?

PrePROD

What is a common class of vulnerabilities that is discovered in PROD due to insecure code creeping in from DEV?

Developer Bypasses

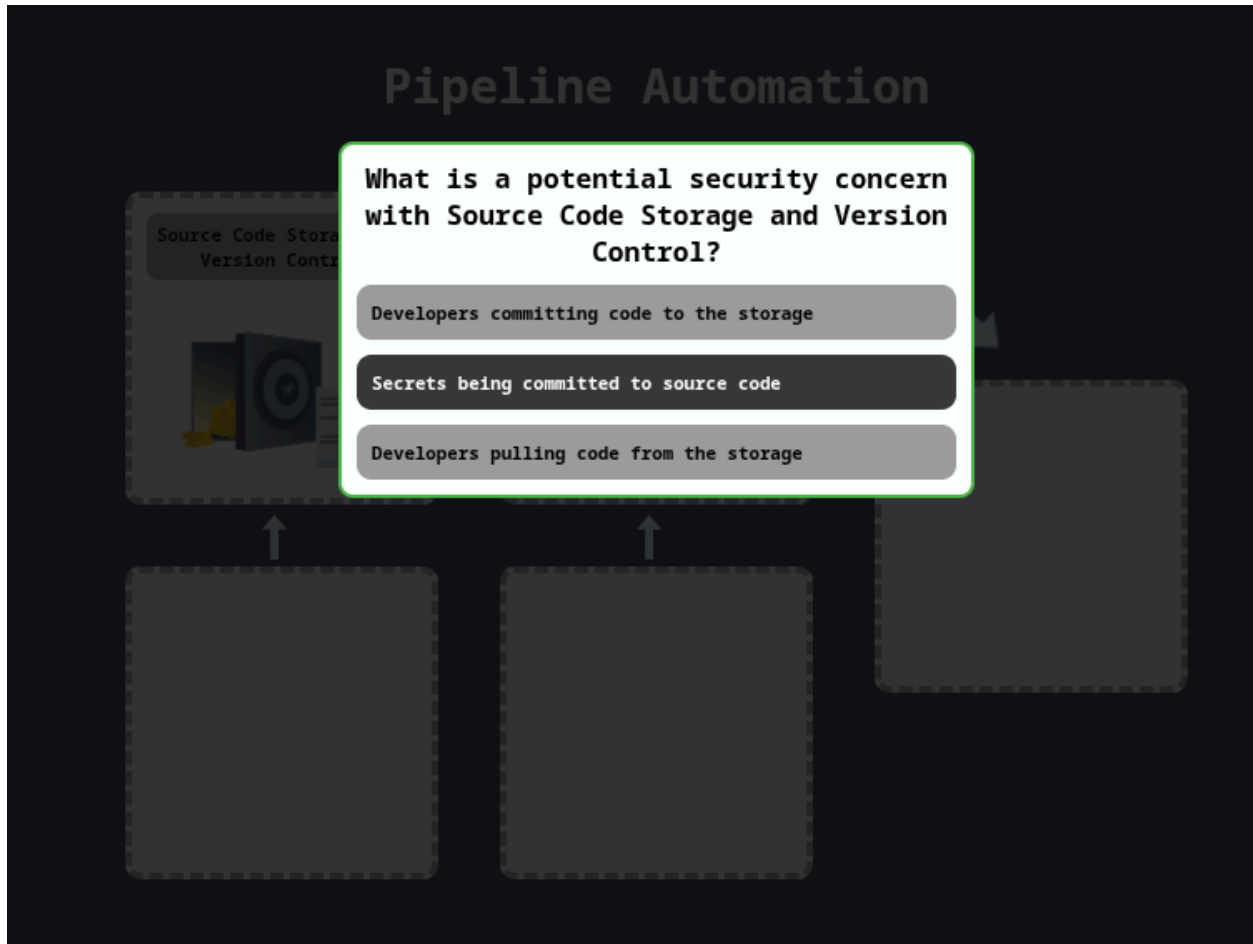
Task 8 - Challenge

Open the site and build your own pipeline to get your flag. Use what you learned to determine which concerns are valid at which stages of the pipeline!

Answer the questions below

What is the flag received after successfully building your pipeline?

THM{Pipeline.Automation.Is.Fun}



Pipeline Automation

What is a potential security concern with Dependency Management?

An outdated dependency that has a known vulnerability is being used

The application having too little dependencies

The dependencies being developed internally



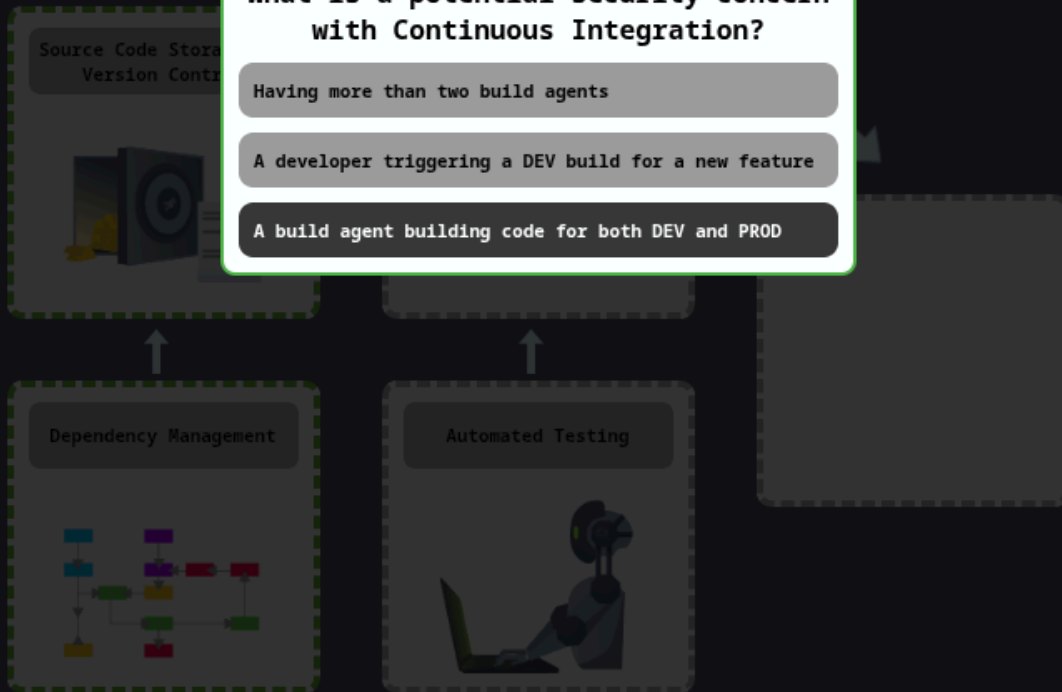
Pipeline Automation

What is a potential security concern with Continuous Integration?

Having more than two build agents

A developer triggering a DEV build for a new feature

A build agent building code for both DEV and PROD



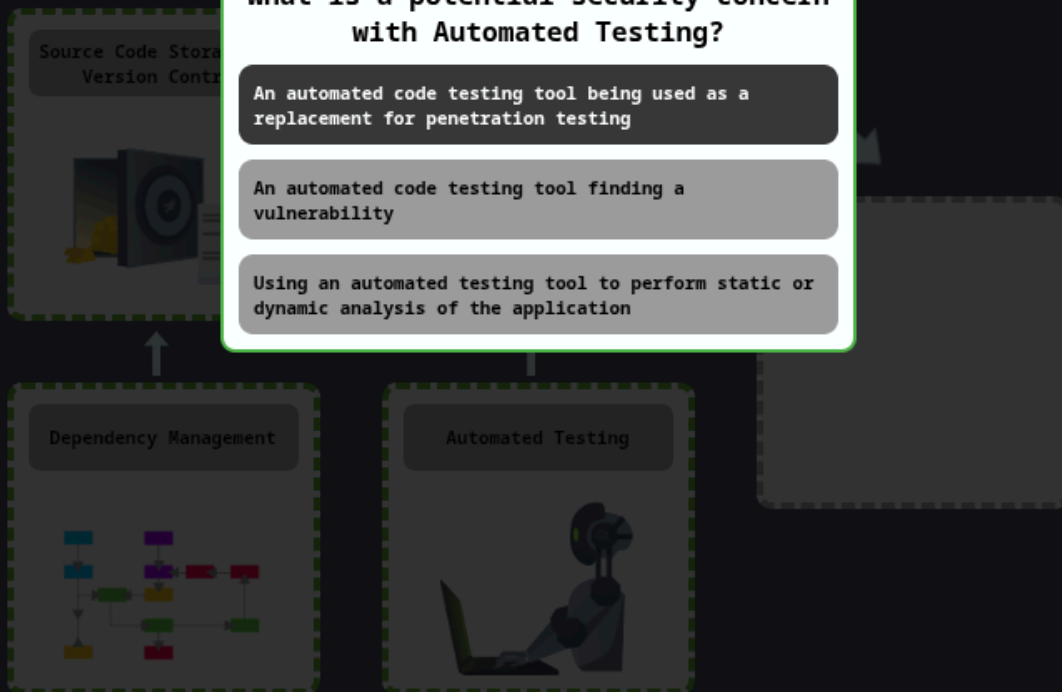
Pipeline Automation

What is a potential security concern with Automated Testing?

An automated code testing tool being used as a replacement for penetration testing

An automated code testing tool finding a vulnerability

Using an automated testing tool to perform static or dynamic analysis of the application



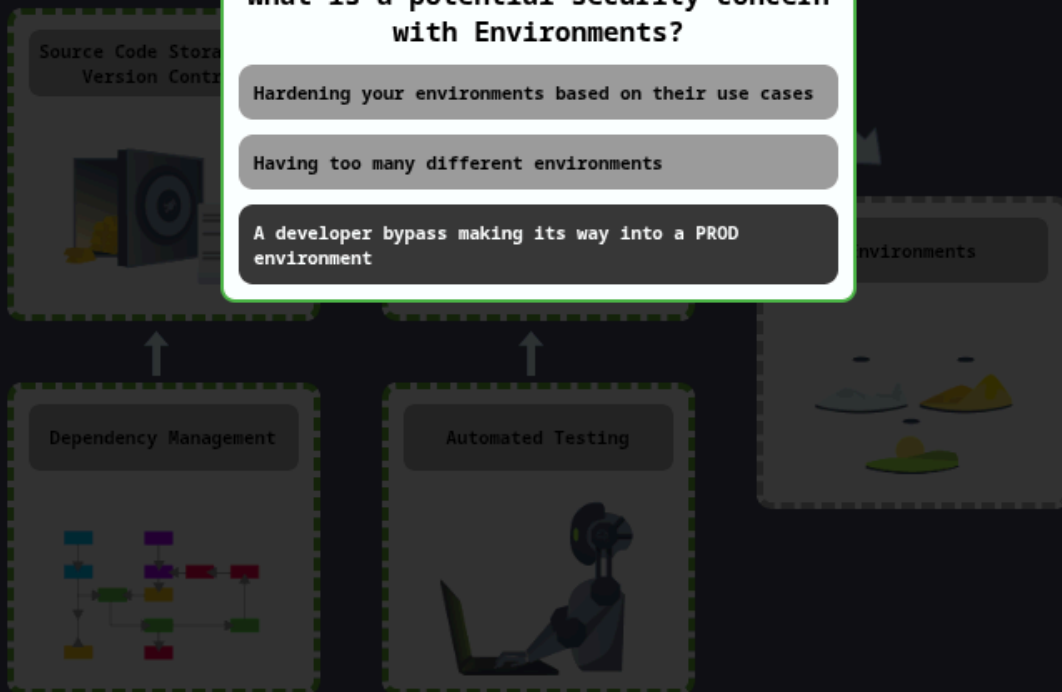
Pipeline Automation

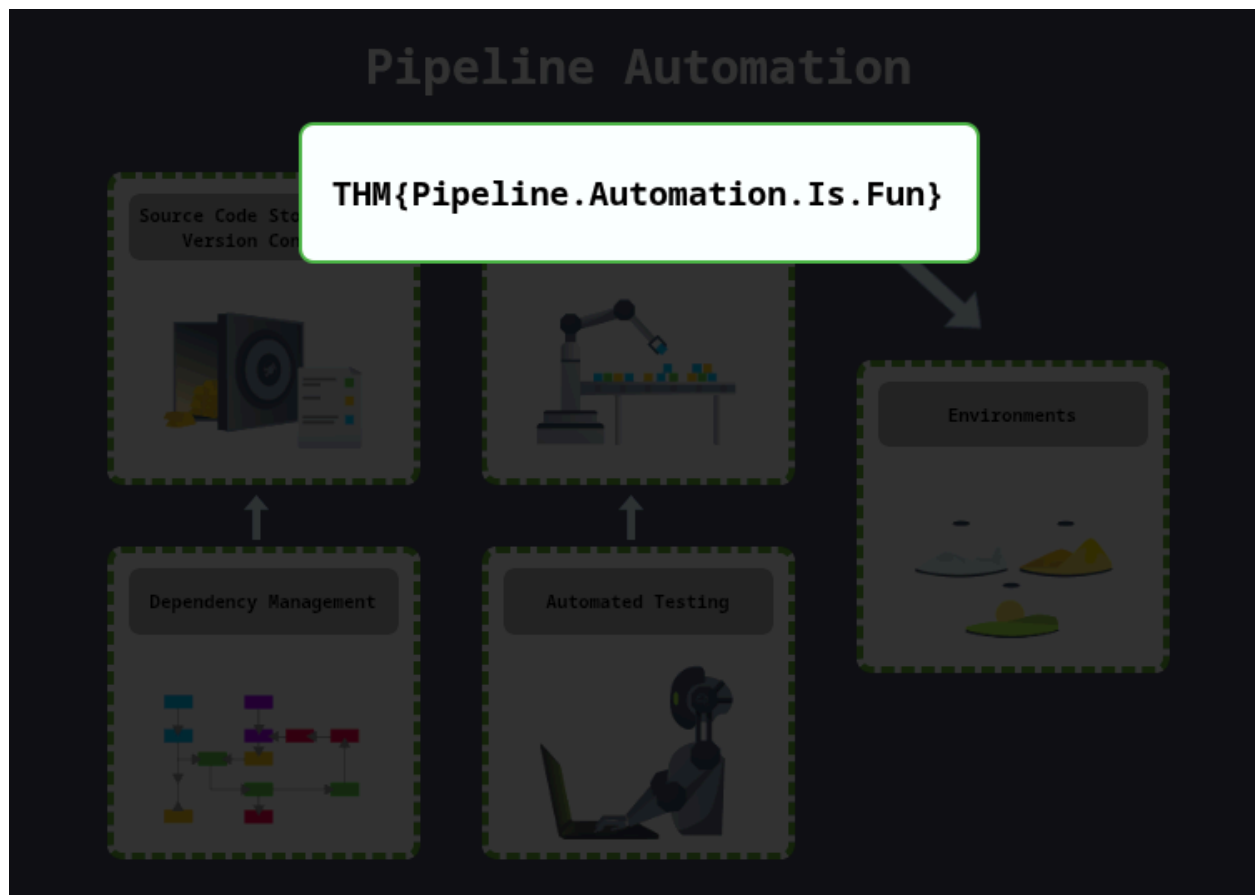
What is a potential security concern with Environments?

Hardening your environments based on their use cases

Having too many different environments

A developer bypass making its way into a PROD environment





Task 9 - Conclusion

Automation in the pipeline has significantly increased the capability of SDLC processes. It has enabled developers to rapidly create and deploy updates to applications. However, these new automation can also lead to an increased attack surface since an attacker can now indirectly attack the application by compromising its pipeline. Implementing secure automation is therefore needed to ensure that the automated pipeline does not increase the risk of application compromise. Throughout the various rooms in this module, we will take a deeper dive into the elements that make up a pipeline and show how security can be applied to each to create a secure, automated pipeline.